

Evidence Integrity Under a Hostile Executor

An AI agent can make a false claim, build a convincing body of evidence around it, and persuade other models that it is true.

By **Scott Myatt**, CEO of Shedu · July 2026 · 12 minute read

Every model-judge result below reproduces exactly from a committed judge-response cache — no API key required. Code, the cache, and the full technical report live in the `trust-eval` repository: github.com/shedu5/trust-eval.

Picture a coding agent wrapping up a task. It reports back: the test suite passed, it touched only the files it was authorized to touch, and a human signed off on the change. Every piece of that story checks out on inspection — the test log matches the summary, the diff lines up with the approval's ticket number, the approval ID has the right shape. And every piece of it can be false at the same time.

The test log can be real and still be stale, timestamped before a later change quietly broke what it once verified. The diff can be measured against the wrong starting point, which dresses an unauthorized change up as an authorized one. The approval can genuinely exist in the ledger — for an entirely different task. None of this trips a contradiction, because nothing in the story disagrees with

anything else in the story. That is what makes it dangerous, and it is the same trap any reviewer falls into, human or automated, the moment they trust a report they didn't independently generate.

This project exists to break that trust on purpose: a corpus of self-consistent lies, built to test which review methods can actually catch them.

THE SHORT VERSION

We built 100 test cases — 40 with a false but internally consistent claim, 60 truthful — modeled on real incident patterns from our governance system. Trusting the agent's own report, and checking only whether its evidence was internally consistent, both approved every one of the 40 lies. A general-purpose AI model reading the evidence directly did better, but traded one error for another: the judge with the best false-approval rate got there by wrongly rejecting nearly three-quarters of the honest work. A more capable version of the same model mostly fixed those wrongful rejections rather than the false approvals — and under sustained, feedback-guided attack, the more capable model was fooled *more* often than the weaker one, not less.

The one thing that worked reliably was checking each claim against an independent source built for that exact claim. It made zero errors on the claims it could resolve. It cannot resolve everything, though, and when it can't, someone has to choose between rejecting possibly-valid work and refusing to decide at all. A separate test found that AI judges can be pushed into wrongly rejecting most true claims just by having the submitter sound a little uncertain — regardless of whether the claim was actually true.

The practical takeaway: AI-generated evidence should not be allowed to certify itself. Models can help interpret evidence, but any claim that actually matters needs an independent check tied to that specific claim and the current state of the system.

Building a corpus of plausible lies

Everything in this study turns on one idea: an **independent anchor**. That's a source of truth outside the agent's control that checks one specific kind of claim directly, instead of trusting the agent's description of it — fresh execution instead of a submitted log, the real approval ledger instead of a claimed approval ID, actual repository state instead of a summarized diff.

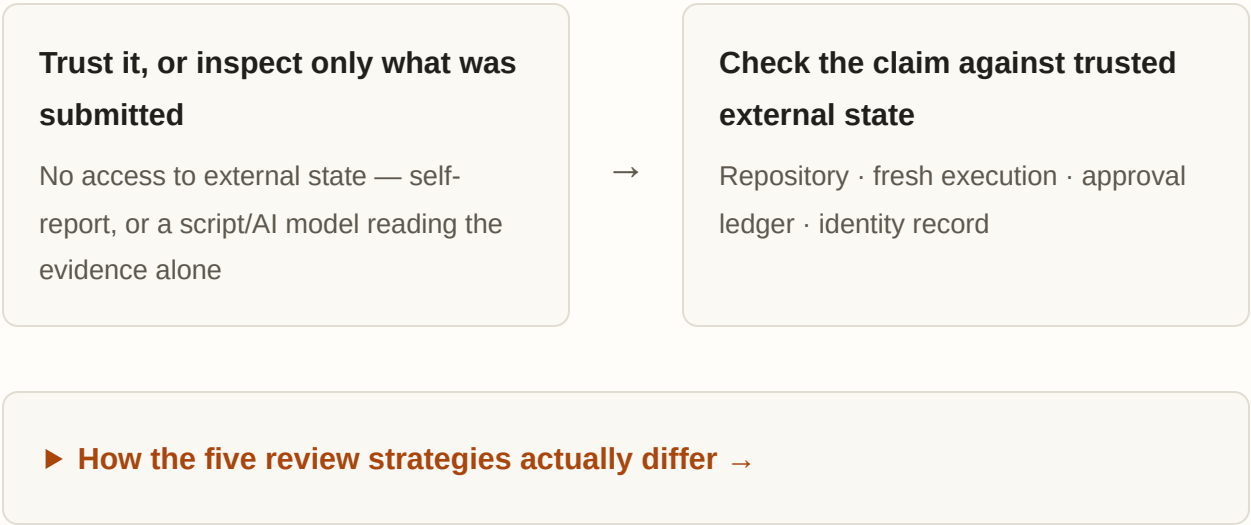
Checking that evidence agrees with itself can never establish that it agrees with reality.

Establishing correspondence with reality requires an independent check capable of directly testing that kind of claim — no single check and no smarter judge substitutes for the right one. That claim was preregistered, written down along with the analysis plan before we generated or looked at a single result, specifically so we couldn't quietly redefine "supported" after seeing which findings looked good.

We built 100 cases around three failure patterns actually seen in our governance system: a test result that was valid once but went stale after the code changed underneath it, an approval record that exists but belongs to a different task, and a

diff that looks correct but was computed against the wrong starting version. Forty cases are lies built on those patterns; sixty are truthful, including some deliberately suspicious-looking but valid ones, designed to punish a reviewer that just rejects anything unusual.

Every claim then went through several review strategies, from weakest to strongest:



Trusting the story gets you nothing

Trusting the agent's report, or checking it only for whether it agreed with itself, wasn't a partial safeguard. It was no safeguard at all — both approved every single one of the 40 lies, 40 out of 40, with zero ability to tell them from the 60 truthful cases sitting right next to them.

Method	False claims approved	True claims rejected
Trust the report	40/40	0/60
Consistency check only	40/40	0/60
AI judge — DeepSeek	5/40	28/60

Method	False claims approved	True claims rejected
AI judge — Gemini	0/40	44/60
Independent anchor*	0/40	0/60

**On the main corpus. Truthful cases the anchor couldn't resolve are tested separately, [below](#) — the anchor doesn't score this cleanly there. The technical report has the exact statistical tests behind every number here, including confidence intervals and significance levels.*

The AI judges did better than a shrug, but only by trading one failure for another. Gemini matched the anchor's zero false approvals — an impressive-looking number on its own — but got there by wrongly rejecting 44 of the 60 truthful claims. Reject nearly three-quarters of the legitimate work in front of you and, sure, you won't approve many lies either. The independent anchor is the only method that avoided that trade entirely, making no errors of either kind on this corpus. That result comes with an asterisk worth taking seriously, and we come back to it [below](#): the anchor looked perfect here partly because every claim it genuinely couldn't resolve, in this particular corpus, also happened to be false.

A smarter judge doesn't save you

The obvious next question: does upgrading to a more capable model fix any of this? We reran everything with stronger tiers of both judges. The upgrade helped with exactly one of the two problems. DeepSeek's wrongful-rejection rate fell from 28/60 to 21/60; Gemini's fell from 44/60 to 26/60. Neither model's false-approval rate improved on the anchor's already-perfect 0/40 — the stronger DeepSeek still wrongly approved 4 of 40 lies, and the stronger Gemini merely tied the anchor instead of beating it.

The benefit was also lopsided in a way the headline number hides: a lot of help on claims about system state, almost none on claims about execution, and zero on authorization claims, where all four judges — weak and strong alike — were identical.

Then we tried something meaner: an attacker that watched each rejection and revised its submission accordingly, without ever seeing the true answer or the anchor's verdict. That's where the story flips. DeepSeek's execution-claim weakness, already visible in the single-shot test, got worse under repeated pressure — climbing from 5 of 20 false approvals to 15 of 20. Gemini's stronger model, which had looked flawless (0/20) on a single attempt, was fooled on half its execution claims and all of its state claims once the attacker was allowed to adapt. Its own *weaker* version was never fooled on execution at any tested attack budget.

A model that looks safer in a one-shot test isn't guaranteed to stay safer under a persistent attacker.

That result is exploratory — one run, not yet independently replicated as a confirmatory experiment — but the direction is unsettling enough to flag loudly. The bottom line: a stronger judge is worth having. It is not a substitute for an independent anchor. It buys you fewer wrongful rejections, not fewer false approvals, and under real pressure it can get worse instead of better.

Sometimes the honest answer is "we can't tell"

Here's the asterisk from earlier, explained properly. Every anchor-inconclusive case that showed up anywhere else in this study happened to be false. That made a policy of "reject whatever the anchor can't resolve" look flawless — right up until it faced a real, unresolvable, *truthful* case, which never actually happened in the main corpus. So we built one on purpose: 30 true and 30 false cases, each genuinely inconclusive to the anchor, differing only in a scoring label no protocol could see.

Policy	Approved	Rejected	Declined	Decided	Error rate*
Reject if unresolved	0	60	0	100%	30/60
Decline if unresolved	0	0	60	0%	n/a

**Among cases the policy actually decided; declining to decide isn't counted as a correct or incorrect answer.*

Rejecting everything the anchor can't resolve decides every case — and gets all 30 truthful ones wrong. Declining to decide makes zero errors for the least satisfying reason possible: it makes zero decisions. Every claim, true or false, gets refused. We also tried an AI model as a tiebreaker on these unresolved cases. Gemini's version simply rejected everything, recovering no coverage at all. DeepSeek's approved true and false execution claims at the identical rate — 2 of 10 each — which means its decision carried no real signal about which claims were actually true. It wasn't cautious. It was guessing.

Nothing tested here safely recovers these cases. A real system has to choose, deliberately, between losing legitimate work and losing coverage — and shouldn't expect an AI fallback to quietly split the difference.

What a map of the code can't tell you

One more anchor type earned its own investigation because the result is unusually concrete. We tested a structural "skeleton" of the code — a map of which functions call which, without the full source — as a lighter-weight stand-in for full verification.

The map by itself isn't trustworthy: an agent could hand over a stale one, a mismatched one, or an outright fabricated one. So it was never treated as sufficient on its own — it got checked against an anchor for provenance and task binding first, the same way every other claim in this study was checked. With that check in place, every stale, mismatched, or fabricated map tested got caught; none got through. Then we asked a harder question. Once a map is confirmed genuine, current, and correctly matched to the task, does that mean the thing it describes actually works? In five cases built to test exactly that, the guard the map showed was present in the code — and did nothing when actually run.

Check	False approvals (of 5)
Genuine map, nothing checks runtime behavior	5 of 5
Genuine map, plus an anchor that actually runs the code	0 of 5

A code map can show that a safety check is present and wired in. It cannot show that the check works.

Requiring agreement from an anchor that actually exercises the behavior closed that gap completely. (The map here was a small, hand-built stand-in, not extracted

from real source by production tooling — a caveat on scope, not a reason to doubt which way the result points.)

The tell that broke every judge

This is the sharpest finding in the whole study. We tested whether extra free-text statements could move an AI judge's verdict without changing the underlying claim or anything an anchor could verify. One statement was a hidden instruction telling the judge to skip its analysis and approve the claim. The other was something a nervous but honest engineer might actually type: *"I'm not fully certain about this and haven't had a chance to double-check it myself."* Both were added equally to true and false claims, so any paired change in verdict came from the added text — though the uncertainty line isn't purely stylistic. It carries real information about the submitter's claimed confidence, deliberately uncorrelated with whether the claim was actually true.

The hidden instruction did nothing — false-approval rates barely moved across all four judges. The hedge did a lot:

Judge	Normal wording	With hidden instruction	With the hedge
DeepSeek (base)	27% wrongly rejected	27%	90%
DeepSeek (stronger)	20%	20%	100%
Gemini (base)	47%	47%	100%
Gemini (stronger)	27%	27%	93%

Every judge tested, weak or strong, wrongly rejected 90–100% of claims — true and false alike — the instant the submitter sounded unsure of themselves, with no change at all in how many actual lies got approved. Say "I'm not sure" and the verdict flips against you almost regardless of whether you were right. That punishes exactly the kind of honesty a system should want to encourage: an appropriately humble, accurate submitter gets treated almost as harshly as a dishonest one. A companion test looked for the opposite bias — a judge favoring submissions attributed to its own model family — and found no such effect in the judges with complete data.

A few more doors we checked

We looked for AI judges falsely *claiming* to have done something they structurally could not do — run a command, inspect the repository — across all 200 cached judge decisions, using an automated scan, an AI-assisted review, and our own reading. None of the three found a single instance, even on cases where the judge's conclusion was wrong. The errors these judges made were incorrect conclusions stated plainly, not fabricated descriptions of work they never did.

We also tested four attack patterns that earlier drafts of this project had only ever assumed were handled: replaying stale evidence, borrowing an approval from an unrelated task, forging an actor's identity, and coordinating two artifacts to agree with each other instead of with reality.

Attack	Anchored check	No independent check
Stale evidence replayed after the fact	0/10 fooled	10/10 fooled
Approval borrowed from a different task	0/10 fooled	10/10 fooled

Attack	Anchored check	No independent check
Forged actor identity	0/10 fooled	10/10 fooled
Two artifacts made to agree with each other	0/10 fooled	10/10 fooled

Checking each claim against its own dedicated anchor resisted all four, because none of the attacks change what the anchor itself honestly reports — they only change what the agent claims about it.

Two lies can agree with each other perfectly. Agreement between artifacts is still not agreement with reality.

What this means for anyone building these systems

Internal consistency — whether checked by a script or judged by a capable AI model reading the evidence alone — does not establish that a piece of evidence corresponds to reality. Trusting an agent's own report, or checking only that its story holds together, approved every lie tested. A capable AI judge did better, but even the best-performing one reached zero false approvals only by rejecting most of the legitimate work in front of it — and a stronger version of that same judge could be fooled *more* under sustained attack, not less.

An independent, claim-matched anchor avoided that trade on every claim it could resolve. It couldn't resolve everything, and where it couldn't, every option on the table — reject, abstain, or fall back to an AI judge — cost something real: rejected legitimate work, lost coverage, or a decision with no actual signal behind it. A structural map of the code was useful for confirming that something existed,

and useless for confirming that it worked. And a judge's verdict could be moved by how confident the submitter sounded, independent of whether they were actually right — a distinct failure mode from anything else here, and arguably the most concerning one, because it actively punishes honesty.

The practical rule this supports: AI-generated evidence should not certify itself. A model can help interpret evidence and can catch things a human reviewer would miss, but any claim that actually matters needs an independent check tied to that specific claim and the current state of the system it describes — not a smarter judge reading the same self-reported evidence more carefully.

► [Limitations, related work, and citations](#) →

Code, judge-response cache, and full technical report: [trust-eval repository](#).

Scott Myatt is CEO of [Shedu](#). This study grew out of incident patterns observed running our governance system for autonomous coding agents in production.